

SEUPD@CLEF: Team SARD on Longitudinal Evaluation of IR Systems Using Language-Aware Processing and LLM-Based Query Manipulation

Damiano Caon¹, Riccardo Dal Maschio¹, Alessandro Disarò¹ and Sofia Maule¹

¹University of Padua, Italy

Abstract

This paper presents the participation of Team SARD in the *CLEF!* (*CLEF!*) LongEval 2025 shared task, which investigates the longitudinal evaluation of information retrieval systems on evolving web collections. In this report we explore a wide range of indexing and querying configurations by varying analyzers, language detection granularity, and query formulation strategies using the tools provided by the Apache Lucene framework. Additionally, we examine the impact of synonym-based query manipulation using external lexical resources. The evaluation is based on standard retrieval metrics: MAP, nDCG, and interpolated precision at standard recall levels, all computed using `trec_eval`. A custom MATLAB-based pipeline was developed to automate metric extraction, aggregation, and visualization across monthly snapshots. Our results indicate that document-level language detection and the use of phrase-based retrieval improve performance in most scenarios. However, the benefits of query manipulation appear limited when compared to the effects of other design choices. We also observe high variability across queries and reflect on the implications for evaluation reliability. Future work will explore learning-to-rank approaches and the integration of large language models, contingent on the availability of higher-quality computational resources and adequate relevance judgments.

Keywords

Information Retrieval, LongEval, CLEF, Query Evaluation, Boolean Query, Phrase Query, Query Manipulation, `trec_eval`, MAP, nDCG, Interpolated precision at standard recall, Language Detection, Lucene, Retrieval Performance, Analyzer Comparison, Longitudinal Evaluation, LLM, `gpt4.0-turbo`

1. Introduction

IR! (*IR!*) systems are often trained on static datasets, but real-world data, especially on the Web, evolves continuously over time. As a result, their effectiveness tends to degrade when applied to new, unseen data. This issue is at the core of CLEF 2025 LongEval Task 1 (Web Retrieval)[1], which focuses on evaluating the robustness of *IR!* models over time.

The aim of this report is to present our solution to this challenge. Our approach was to design an IR system capable of maintaining stable performance across temporal shifts. In particular, we focused on the development of custom language-specific analyzers for the preprocessing of documents and queries, and also the correction of user queries using GPT-4 Turbo. Unlike traditional query expansion methods, which add semantically related terms, our use of GPT-4 Turbo aims to rewrite and clarify ill-formed or noisy queries without altering their intent, resulting in more precise and effective matching with the indexed content.

The paper is organized as follows: Section 2 describes our approach; Section 3 explains our experimental setup; Section 4 discusses our main findings; and Section 5 draws some conclusions and outlines directions for future work.

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2024/2025

✉ damiano.caon@studenti.unipd.it (D. Caon); riccardo.dalmaschio@studenti.unipd.it (R. D. Maschio); alessandro.disaro.1@studenti.unipd.it (A. Disarò); sofia.maule@studenti.unipd.it (S. Maule)



© 2025 Copyright for this paper by its authors.

Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

2. Methodology

The methodology employed in this work follows the principles of *iterative design*. An initial baseline pipeline was defined at the beginning of the system development. As the work progressed, this structure was continuously refined through a cycle of implementation, testing, and improvement.

These iterations were not carried out blindly or based solely on trial and error; rather, they were guided by reasoned decisions aligned with established theoretical findings in the field of Information Retrieval. This approach enabled the progressive enhancement of the system based on both practical outcomes and theoretical grounding.

The architecture of the system, which also served as the foundational pipeline, was composed of the following key components:

- **Parser** – responsible for reading and structuring the raw input data;
- **Analyzer** – performed preprocessing tasks such as tokenization, stop-words removal, and stemming;
- **Indexer** – constructed and maintained the index used for information retrieval;
- **Searcher** – executed queries against the index and returned ranked results.

In order to manage the execution of the pipeline, a lightweight `Main` class was developed. This class was designed to maximize operational flexibility, allowing the user to control the system with the greatest degree of freedom possible. For instance, it is possible to select and process only a subset of months by modifying the initial month list, or to choose whether to perform indexing, searching, or both, by simply adjusting internal configuration flags. Additionally, users can easily set parameters for the search process, such as the number of documents to retrieve and the type of query to execute – either boolean or phrase queries. This design choice proved particularly valuable during system evaluation, as it avoided redundant indexing, a computationally expensive step, and enabled rapid testing of alternative configurations on previously built indexes.

The previously listed modules were implemented as standalone packages capable of interacting seamlessly with one another. Each component was designed with modularity in mind, allowing for flexible configuration and easy parameter adjustments. This architectural choice facilitated experimentation with different setups during the evaluation phase, supporting the iterative and theoretically-informed development process.

2.1. Parser

The Parser is responsible for reading the raw JSON files and transforming them into structured documents suitable for subsequent analysis and indexing. Although the component is relatively straightforward, particular care was taken to ensure robustness and efficiency during the design and implementation phase.

To process potentially very large datasets without running into memory limitations, the Parser was implemented using a streaming approach. Specifically, it employs a tokenized JSON reader (based on the Jackson library [2]) that reads the documents sequentially from the file. This design allowed the system to handle thousands of documents efficiently, without loading the entire dataset into memory.

During the parsing process, each document is cleaned to reduce textual noise and standardize the content before further processing. The following preprocessing steps are applied to the textual contents:

- Removal of emojis;
- Removal of URLs;
- Removal of HTML tags;
- Conversion to lowercase.

These cleaning operations were motivated by the need to simplify the input text and enhance the effectiveness of downstream components such as tokenization and indexing. In particular, removing highly variable or non-textual elements such as emojis, URLs, and HTML markup helped improve the consistency of the documents without sacrificing meaningful content.

Additionally, the Parser provides configurable options to selectively enable or disable the removal of emojis, URLs, and HTML tags. This feature was designed to offer flexibility during experimentation phases, even though in the final pipeline all cleaning steps were consistently applied.

To ensure robustness, the Parser incorporates a safety check that automatically skips invalid documents, defined as records missing either the identifier or the contents field. This prevented pipeline failures and minimized the impact of noise or inconsistencies in the raw data.

The documents produced by the Parser are instances of the `JsonDocument` class, a lightweight data model encapsulating the `id` and `contents` fields of each document. For consistency across modules, the field names (`id` and `contents`) are defined as constants within the `JsonDocument.FIELDS` inner class, ensuring compatibility with components such as the Analyzer and the Lucene-based indexing system.

Overall, the Parser contributed to establishing a clean and reliable document stream, serving as a solid foundation for the subsequent stages of the information retrieval pipeline.

2.2. Analyzer

To accommodate the multilingual nature of the dataset and support rapid experimentation, we developed a flexible analyzer architecture. The process began with an exploratory component (`TestAnalyzer`) and led to the definition of two final language-specific analyzers tailored to English and French documents.

2.2.1. TestAnalyzer

Initially, due to the presence of documents written in both English and French, we decided to build two separate analyzers—one for each language—so that the most appropriate processing could be applied based on the document’s language. To support rapid experimentation with different configurations, we developed the `TestAnalyzer` class.

This component is designed to be flexible and fully configurable through external JSON files, which define all processing parameters without requiring any code modifications.

Each configuration file specifies the following parameters:

- `tokenizerType`: the type of tokenizer to use;
- `minLength` - `maxLength`: optional token length boundaries;
- `useEnglishPossessiveFilter`: whether to apply the English possessive filter;
- `stopListFilePath`: path to the custom stopword list file;
- `stemFilterType`: the type of stemming filter to apply;
- `language`: the language of the input text;

The `language` parameter also enables language-specific processing rules, such as the handling of contractions in French or possessive forms in English, and selects the most appropriate stemming algorithm.

The `TestAnalyzer` played a central role during the experimental phase, allowing us to evaluate various combinations of tokenization, stopword removal, and stemming for both English and French collections.

2.2.2. FrenchTextAnalyzer and EnglishTextAnalyzer

We selected the best-performing setup and formalized it into two dedicated analyzers: `FrenchTextAnalyzer` and `EnglishTextAnalyzer`. Each analyzer was designed to suit the linguistic characteristics of its respective language, while sharing a common processing pipeline structure.

- *Tokenization*: Both analyzers begin by breaking input text into tokens using Lucene's `StandardTokenizer`, which handles basic word segmentation.
- *Normalization*: A series of filters are then applied to standardize token formats:
 - `LowerCaseFilter` converts all characters to lowercase.
 - `ICUFoldingFilter` removes accents and diacritical marks to harmonize character encodings.
 - `PatternReplaceFilter` eliminates punctuation using a regular expression.
 - `LengthFilter` retains only tokens between 2 and 20 characters, removing overly short or long terms.
- *Stopword Removal*: Language-specific stopwords lists are applied using `StopFilter`. We used two separate lists: `french_stopwords.txt` and `english_stopwords.txt`, both taken from Oracle's default stoplists¹.
- *Stemming and Language-Specific Filters*:
 - The `FrenchTextAnalyzer` includes an `ElisionFilter` to remove contractions like "l'", "d'", and "c'", which are common in French. For stemming, it uses `FrenchMinimalStemFilter`, which we found to perform best by balancing morphological reduction with semantic retention. We also tested `FrenchLightStemFilter` and the `SnowballFilter` with `FrenchStemmer`, but they were less effective in preserving useful lexical distinctions.
 - The `EnglishTextAnalyzer` applies an `EnglishPossessiveFilter` to strip possessive suffixes (e.g., "'s"), reducing noise from possessive forms. It then uses `EnglishMinimalStemFilter`, which outperformed alternatives like `PorterStemFilter` and the `SnowballFilter` with `EnglishStemmer` by avoiding overstemming and maintaining semantic clarity.

Overall, this dual-analyzer setup allowed us to tailor preprocessing to the specific needs of each language, improving indexing consistency and retrieval effectiveness.

2.3. Indexer

The Indexer is responsible for creating the index used for document retrieval. In the final versions of our systems, each Lucene `Document` in the index contains five fields: the document ID, the content (if in English), the content (if in French), the language of the document, and the name of the JSON file from the collection where the original document is located. Having two separate content fields allowed us to apply different analyzers to each, effectively handling both languages. Although it is a fairly standard component, with limited impact on the overall effectiveness of the system, we focused on improving its performance to facilitate the system testing phase. In particular, we concentrated our efforts on two key aspects: multithreading and efficient language detection.

To reduce the total time required to index all documents, and inspired by the work of a previous team participating in the LongEval challenge [3], we designed the Indexer to leverage multiple processors using a multithreaded approach. This optimization resulted in a 50% reduction in indexing time compared to the single-threaded implementation.

¹<https://docs.oracle.com/en/database/oracle/oracle-database/21/ccref/oracle-text-supplied-stoplists.html#GUID-5DAF7499-5EBA-41E6-AF1A-C3BD2C08F88F>

Since the Indexer is also responsible for selecting the appropriate Analyzer based on the document's language, we tried to make the language detection process both fast and reliable. Initially, we considered using the Lingua library [4], a widely adopted Java tool for language detection, but we observed that it introduced a significant time overhead. Consequently, we implemented a custom language detection class using a hybrid approach, analyzing only the beginning of each document to minimize processing time. The first version of this class was developed to recognize the language of a sentence, but then it was modified to work with full documents. The steps of our language detection process are as follows:

- If the document contains enough words, we use two predefined stopwords lists (one for English and one for French) to estimate the language, counting the occurrences of stopwords from each list;
- If the resulting confidence score, i.e. the difference in the counts divided by the total number of stopwords occurrences, exceeds a predefined threshold, the detected language is returned;
- If confidence is too low or the text is too short, we fall back to the Lingua library, restricting detection to English and French;
- If the result is UNKNOWN (e.g., if the text is in a different language), we default to French.

After experimenting with various parameter settings, we selected the following configuration:

- A minimum of 10 words to attempt detection using the stopwords approach;
- A maximum of 500 words to process per document;
- A confidence threshold of 0.2 for the stopwords-based detection.

This configuration was tested on approximately 46000 documents, corresponding to the first three JSON files from the September 2022 snapshot. The results showed that in roughly 99% of the cases, the language detected by our hybrid method matched the one identified by Lingua when applied to the full text. Furthermore, a manual review of the mismatches revealed that in almost 60% of the cases, the discrepancy was not actually a misclassification, as the texts were multilingual or written in a different language altogether. The manual review also helped us better understand common sources of detection inconsistency, such as:

- Mixed French and English texts (e.g., doc141);
- Lists from e-commerce sites or product catalogs (e.g., doc1354517);
- Documents written in various languages and sometimes different alphabets (e.g., doc258);
- Cookie banners or browser-related boilerplate (e.g., doc3760).

Despite a slight loss in accuracy compared to the full-text Lingua approach, our hybrid method achieved a 95.6% speedup in language detection, leading to a significant overall reduction in indexing time. It is worth mentioning that the language detection class also contains a method to recognize the language of single words by checking two large vocabularies (one for French and one for English), with a fallback on Lingua if the term is not found there. However, this method proved unreliable, often failing with complex or uncommon terms, and was therefore not used in the final systems.

Finally, by inspecting the collection, we noticed that some documents were repeated multiple times within the same snapshot (e.g., doc3173607 appears 56 times in the September 2022 snapshot). To address this, the Indexer was modified to check whether each document had already been processed before indexing it.

2.4. Searcher

The Searcher component is responsible for handling the search functionality in the information retrieval system. It is designed to query the index, process the results, and output the relevant documents based on the provided queries and QRELS. The implementation supports various features and settings

to try different approaches and improve or comparing retrieval processes, such as query expansion, handling multiple languages (English and French), and parallelization to improve (time) performance. The configurability of the Searcher object has allowed us to easily swap from a setup to the other, in order to try as many configurations as possible, using different types of analyzers with different query types.

2.4.1. Query Processing

The search process begins by reading the set of provided topics: a custom `QueryReader` object is used by the `Searcher` class to parse the queries of a certain month and converting them in `QualityQuery` objects, for mapping reasons, which can easily be processed by the system. For each of these objects, a query based on the provided content, either in the form of a Lucene's `BooleanQuery` or a `PhraseQuery`, depending on the configuration, is created. The `BooleanQuery` is particularly useful for basic matching, where the query terms are connected using logical operators, such as `OR`. On the other hand, `PhraseQuery` is employed when the order of the terms in the query is important, allowing for a better matching of phrases rather than individual terms. The `Searcher` object then also accepts two analyzers, both for French and English, which hopefully have to be the same ones used to index the corpora, to guarantee the best match based on the detected language of query terms. Language detection, though, proved to be a challenging factor that impacted the search system's performance. Different systems exhibited inconsistent results depending on the configurations used. Even though we developed a custom `FrenchEnglishDetector`, with better performances in terms of time, the `Lingua` [4] `LanguageDetector` seemed to perform better in average, even if it was slower.

2.4.2. Parallelization

To enhance the efficiency of the search process, especially to handle the large indexes and a big number of queries, the `Searcher` has been designed to utilize parallel processing. The search tasks for each topic are executed in parallel using Java's `ExecutorService`. This approach enables the system to process multiple queries simultaneously, reducing the overall search time and improving scalability. The number of threads used for parallelization is configurable, with a default that matches the number of available processors on the machine. This ensures optimal resource utilization and allows for fine-tuning based on system specifications.

To have an indicative measure of how much the performance improved, the searching was performed roughly 70% quicker with 4 threads instead of 1.

2.4.3. QueryExpansion

Another challenging aspect has been query expansion. The basic process was to generate synonyms from the query terms and analyze them to create the stems to build the queries to perform on the index. This way we hoped to increase the potential matches for every query. For the `BooleanQuery` object the process was simply adding synonyms to them (with the `OR` clause). However, we immediately noted a drastic fall in performance even when adding just a handful of synonyms to the queries. We therefore decided to reduce that number as much as possible and to boost, or more accurately, to penalize the strings not derived directly from the query terms, using Lucene's `BoostQuery` object. It is important to note that even boosting (or not boosting) a single term with a factor of 1 worsened performance. Regarding the `PhraseQuery` objects, a simple way of enhancing queries has been to use the `slop` parameter to generate permutations of the starting query. In addition, to better manage more complex query expansion with synonyms, we decided to use a Lucene object designed specifically for this application, the `MultiPhraseQuery` class.

Unfortunately, as can be seen in Section 4, these procedures did not help improve performance.

2.4.4. GPT elaborated queries

To improve the quality of the queries, we processed them using GPT-4.0 Turbo asking the LLM to restore proper diacritics, correct misspellings, and rewrite overly long queries. In a real IR system, this processing would typically be done at runtime each time a new query is submitted by a user. However, since we needed to handle a large number of queries, we chose to process them in advance to avoid repeating the same procedure at every run (also considering that the APIs for the chosen LLM are not free). Since the snapshots from June 2022 to September 2022 contained numerous queries related to adult content, which caused problems when processed by GPT, we decided to collect all queries from the remaining training set, remove duplicates, and improve only those. As a result, queries from October 2022 to February 2023 were fully rewritten, while for the earlier months the proportion of rewritten queries ranged from 78% to 88%.

2.4.5. SynonymsGenerator

To perform query expansion, we developed a class capable of returning a list of synonyms for a given English or French word. Our original idea was to accomplish this using two different techniques: one based on fixed dictionaries and the other leveraging LLMs.

For the dictionary-based approach, we used WordNet [5] for English queries and WOLF [6], its French counterpart, for French queries. While we had a CSV version of WordNet readily available, we had to pre-process the WOLF XML file to extract synonym lists for each word. At runtime, the extracted synonym sets are parsed into dictionaries, and when the proper methods are called, the SynonymsGenerator provides a list of synonyms for the given word. This approach is very fast, but the generated synonyms are not selected with regard to the context of the query, often leading to query drift and poor effectiveness. Additionally, there were some strange links in the dictionaries, resulting, for example, in *gondola* being provided as a synonym for the English term *car*, or *clientélisme* for the French term *voiture*.

To improve the quality of the synonyms, we first attempted to build a local Python server using Flask and the model LLaMA-3.2-1B-Instruct, but this approach proved unsuccessful. Despite carefully designed prompts explicitly instructing the model to return only the expanded query, the output was often inconsistent or poorly formatted. For instance, when expanding the French query:

```
évolution voiture
```

the model returned:

```
évolution de la voiture, évolution automobile, évolution du véhicule,  
évolution des voitures, avancement technique, amélioration continue,  
innovation technologique, avancée scientifique, progrès industriel,  
développement numérique, amélioration continue de la voiture, avancement  
technologique, progrès de l'automobile, évolution de l'industrie
```

which was satisfactory in terms of format but slightly too broad in terms of relevance. However, for the English version of the same query:

```
car evolution
```

the model produced the following output:

```
Original Query:\n Car evolution refers to the process of changing a  
vehicle's design, features, and technologies over time.\n Synonyms/variants:\n1. Automotive evolution\n2. Vehicle development\n3. Car transformation\n4. Motor vehicle evolution\n5. Vehicle modernization
```

despite the prompt being identical in structure and clearly requesting a clean list of synonyms in plain text, without any preamble or line breaks. Furthermore, the server required nearly 15 seconds to process each request, making it impractical for handling the large number of queries provided in the LongEval challenge. Similar problems were encountered when trying GPT-3.5 Turbo, whereas they disappeared when testing with GPT-4.0 Turbo. Unfortunately, the cost of using the latter model's APIs was prohibitive given the volume of queries to be processed, and the use of LLMs for synonym generation was ultimately abandoned.

2.5. Experimental Strategy

Our experimental process was structured in multiple stages to explore the impact of different indexing and querying configurations on retrieval performance.

Given the multilingual nature of the documents, we initially performed runs using sentence-level language detection, where each sentence in a document was analyzed individually to assign language-specific analyzers. This approach ensured that documents containing both French and English sentences (often due to browser-related boilerplate) were processed with the appropriate analyzers for each part. In the first phase, we tested all available analyzer configurations using `BooleanQuery` in the searcher, fixing the maximum number of documents retrieved per topic at 300, to identify the most effective configuration. The results showed that the best analyzer was `minAnaBool11` (*Subsection 4.3.1*), so all subsequent steps were carried out using this configuration.

Next, we aimed to increase indexing speed by switching to document-level language detection, where the entire document was assigned to either the English or French analyzer based on a global analysis, instead of detecting the language at the sentence-level (based on a sample of 1000 documents, the average number of sentences per document was approximately 30). As expected, this change reduced indexing time and, surprisingly, also improved retrieval performance (*Subsection 4.3.2*).

To enhance precision, we tested the same analyzer using `PhraseQuery` to prioritize retrieving documents where query terms appeared close to each other, allowing for a small margin to avoid overly strict matching. A comparison between the previous `BooleanQuery` run and the new `PhraseQuery` run confirmed the increase in effectiveness (*Subsection 4.3.3*). We also increased the maximum number of retrievable documents to 1000 for both `BooleanQuery` and `PhraseQuery`, observing only a negligible gain in effectiveness but a notable increase in the number of relevant documents retrieved with `BooleanQuery` (an average of 32%). As a result, we decided to set the maximum retrievable documents to 1000 for `BooleanQuery`, while keeping it at 300 for `PhraseQuery`, since the increase in search time was not justified by the marginal gain in recall.

Recognizing that Qwant users often omit diacritics and that some queries contained errors or were poorly structured, we applied LLM-based query correction as described in *Subsection 2.4.4*. The results showed that queries re-elaborated by `gpt4.0-turbo` improved performance for `BooleanQuery` but slightly decreased it for `PhraseQuery` (*Subsection 4.3.4*).

To further increase recall, we tested query expansion as described in *Subsection 2.4.5*, but this approach backfired: not only did we retrieve fewer relevant documents, but we also observed a drop in MAP compared to the baseline, particularly for `PhraseQuery` (*Subsection 4.3.5*). This was likely due to the poor quality of the `SynonymsGenerator`, which led to query drift. Analyzing the topic-level MAP from the December 2022 run, we observed that:

- 4227 queries (55%) were identical regardless of the introduction of query expansion;
- 1221 queries (16%) improved, with an average increase of 0.0542;
- 2213 queries (29%) worsened, with an average decrease of 0.0399.

This indicates that while some generated synonyms were useful, most were not. Note also that we used the GPT-elaborated queries for PhraseQuery as well, due to the structure of the SynonymsGenerator, which favors words with proper diacritics.

Finally, we sought to combine the strong precision of PhraseQuery with the broader recall of BooleanQuery, performing both approaches together and applying different boost values when combining them. We tested combinations both with and without GPT-elaborated queries, based on what we had learned about their respective effects. The best results were achieved using boost values of 1.5 for PhraseQuery and 0.9 for BooleanQuery, with GPT-elaborated queries. This combination yielded the highest MAP, nDCG, and number of relevant documents retrieved (*Subsection 4.3.6* and *Appendix ??*).

3. Experimental Setup

3.1. Collection

The experiments were conducted using the LongEval-Web Retrieval collection² (downloaded prior to the last modifications on 05/05/2025), a benchmark specifically designed to study the robustness and stability of web search engines over time. The collection aims to address two fundamental questions in IR!:

- How does a search engine behave as the underlying collection of documents evolves?
- When and how frequently should an IR! system be updated to maintain performance as the document collection changes?

The queries were created in French and extracted from real user logs, while the documents may contain content in multiple languages, primarily French.

The dataset is organized into 15 monthly snapshots (June 2022 – August 2023), which enables fine-grained evaluation of IR systems over time. Compared to previous LongEval editions (2023 and 2024), the 2025 iteration of the collection features an expanded set of snapshots, allowing for more detailed analysis of collection dynamics.

The dataset is composed of:

- **Training set:** nine monthly snapshots from June 2022 to February 2023, containing approximately 18 million documents, more than 13,000 queries, and associated relevance judgments (qrels);
- **Test set:** six monthly snapshots from March 2023 to August 2023, including documents and queries, still to be released.
- **Relevance judgments** were derived from implicit user feedback, collected automatically via a click model based on Dynamic Bayesian Networks trained on Qwant search logs. The model outputs a probability of document attractiveness, which is then mapped to a 3-point relevance scale (0 = not relevant, 1 = relevant, 2 = highly relevant). This automatic estimation process is scheduled to be complemented with explicit human relevance assessments after the official submission deadline.

Overall, the full collection comprises fifteen distinct datasets, each corresponding to a specific monthly snapshot.

During the development of this project, we observed a significant performance gap between the first four months of the training collection and the subsequent five. To better understand the reasons behind this, and with the hope of finding ways to improve our systems' scores, we spent considerable time analyzing the provided documents, queries and qrels. We wrote some Python scripts to speed up

²<https://clef-longeval.github.io/data/>

frequent tasks, such as retrieving a document by its ID, and manually examined the files to identify potential issues. We discovered that, at least for some queries, there is a clear mismatch between the queries file and the qrels file. For example, query ID 1 for June 2022 refers to an adult content website, but the documents marked as highly relevant in the qrels were about World War I (which actually corresponds to query 3). A similar situation occurred with query 27, adenovirus, which was associated in the qrels with a document about energy efficiency. Moreover, although the LongEval challenge website states that filters were applied to exclude adult content from the collection, it was not the case for the first months, possibly explaining the cause of these mismatches. Further analysis revealed that even the last five months of the training collection were not entirely immune to this problem, although the impact there was smaller. In any case, this manual review was crucial in helping us decide which approaches to pursue and which to abandon as using certain methods would have been misleading without reliable qrels to serve as ground truth.

3.2. Measures

To assess the effectiveness of our retrieval system, we adopted three widely used evaluation metrics: **MAP!** (*MAP!*), **nDCG!** (*nDCG!*), and interpolated precision at standard recall levels (iPrec@Recall). These metrics were selected because they capture complementary aspects of retrieval quality:

- **MAP!** measures the overall precision by averaging the precision scores at the ranks where relevant documents occur. It reflects how well the system retrieves all relevant documents.
- **nDCG!** evaluates the quality of the ranking by assigning higher importance to relevant documents that appear earlier in the results list, which is especially important in web search scenarios.
- Interpolated precision at standard recall levels (iPrec@Recall) provides a global view of system performance by reporting the highest precision achieved for any recall level greater than or equal to a given threshold. This metric is a well-established tool for analyzing the trade-off between precision and recall in a smooth and interpretable way.

The evaluation was performed using `trec_eval`³, a standard tool for information retrieval benchmarking. We used it to compare our system’s output (run files) against the official relevance judgments (qrels) provided by the LongEval organizers [1]. This ensured that our results followed a consistent and widely accepted evaluation protocol.

3.2.1. MATLAB

To support the analysis of our retrieval experiments, we developed a set of modular MATLAB scripts that convert the output files generated by `trec_eval` into structured matrices suitable for evaluation and visualization. These scripts form a core part of our workflow, enabling consistent analysis and comparison of results across a large number of runs and metrics, allowing us to draw conclusions over the effectiveness and performances of our trials.

Each run produces one file per metric and month. The scripts parse these files, align the data across queries or recall levels depending on the metric, and consolidate the scores into uniform matrices. For query-level metrics such as **MAP!** and **nDCG!**, the matrices use queries as rows and months as columns. For interpolated precision at standard recall (iPrec@Recall), rows represent recall thresholds. The framework supports automatic loading, aggregation, and visualization of evaluation data, including mean trends, standard deviations, and optional 3D plots (Appendix ??). Thanks to its flexibility and automation, it allowed us to efficiently process and analyze a wide range of system configurations with minimal manual effort.

In addition to the previously described scripts, we also implemented a tool for conducting statistical significance testing across systems. This script performs a two-way **ANOVA!** (*ANOVA!*) over monthly

³trec_eval repository: https://github.com/usnistgov/trec_eval

mean scores and generates multiple-comparison plots to highlight pairwise differences in performance. It supports both **MAP!** and **nDCG!** metrics. The following sections describe each script in more detail.

Matrix Construction from `trec_eval` Output

This script processes the raw output files generated by `trec_eval` and converts them into structured MATLAB matrices suitable for further analysis. It supports a variety of evaluation metrics—such as **MAP!**, **nDCG!**, and interpolated precision at standard recall levels (`iprec_at_recall`)—which can be easily specified via a configuration parameter at the beginning of the script.

For typical query-based metrics (like **MAP!** or **nDCG!**), the script iterates over one file per month, extracts the score for each query, and constructs a matrix where rows correspond to queries and columns to months. Query IDs are aligned across files, and any missing scores are filled with NaN values to preserve dimensional consistency. For metrics like `iprec_at_recall`, which are not tied to individual queries but computed globally over fixed recall levels, the script builds a matrix with precision at recall levels as rows and months as columns.

Once constructed, the matrix is saved in a `.mat` file under a standardized filename pattern. This flexible and modular structure allows new metrics or runs to be processed with minimal changes, enabling reproducible and efficient evaluation across a wide range of system configurations.

Comparative Analysis Across Systems

This script performs a comprehensive comparative evaluation across multiple system configurations by automatically loading their corresponding evaluation matrices for **MAP!**, **nDCG!**, and interpolated precision at standard recall levels (`iPrec@Recall`). For each metric, it computes average values across queries (or recall points) and generates plots that visualize performance over time or across recall levels.

To support multiple levels of insight, the script produces two types of plots for each metric: one displaying only mean values, and another with grouped error bars that include standard deviation. The mean-only plots are particularly useful for identifying performance trends and differences between systems, while the error bar plots provide additional context on variability and stability.

A key feature of this version is its flexibility: users can selectively enable or disable specific plots using toggle flags defined at the top of the script. Systems to be compared are specified as a simple list, and the script handles all data loading, sorting, and plotting automatically. This modular structure allows for fast experimentation and clear visual feedback even when working with a large number of runs and evaluation metrics.

Detailed Run-Level Evaluation

This script performs an in-depth analysis of a single run. It loads the corresponding **MAP!** and **nDCG!** matrices and computes both the mean and standard deviation of query scores over time. These are then visualized using line plots with error bars, offering insights into the variability and consistency of system performance across months.

Additionally, the script includes a 3D visualization where the evaluation matrix is plotted as a surface over the query-month grid. This allows for an intuitive inspection of patterns, such as drops in performance for specific query groups or months. While not always included in the final report for sake of brevity, these visualizations proved valuable during the exploratory phase of the project.

Two-Way ANOVA and Pairwise Comparison

This script performs a two-way ANOVA on the performance scores of multiple systems using monthly averages. This choice is consistent with the rest of our evaluation pipeline, which is structured considering temporal snapshots.

The script supports both **MAP!** and **nDCG!**, and automatically loads the corresponding matrices for a user-defined set of systems. After running the ANOVA, it performs a multiple-comparison analysis to identify statistically significant differences. For each metric, it reports the p-value associated with the system factor and generates visual comparisons that group systems according to statistical similarity. System names are shown in the order they were declared, ensuring readability.

3.3. Repository and Project Structure

The full source code of the project is available at the following Bitbucket repository:

<https://bitbucket.org/upd-dei-stud-prj/seupd2425-sard/src/master/>

The repository contains all scripts and resources needed to reproduce the indexing, retrieval, and evaluation process described in this report.

3.4. Hardware

To perform the experimentations, multiple hardware setups were used. Due to the high computational cost and long execution time of the indexing process, especially when testing different analyzers, the group decided to distribute the workload among its members. Each member was responsible for indexing the dataset using a specific analyzer on their personal machine. To maintain consistency, each member also conducted the corresponding search experiments on the index they had created, using a searcher configured with the same analyzer setup. This ensured the validity of the results while balancing both the indexing and searching load across the group. The table below summarizes the hardware specifications of the systems used:

Table 1

Hardware Specifications Used for Indexing and Searching

Member	Operating System	CPU	RAM	Storage
Damiano Caon	Windows 11	i5-1145G7	16 GB	SSD 1 TB
Riccardo Dal Maschio	Windows 11	i7 8 th gen	16 GB	SSD 512 GB
Alessandro Disarò	Windows 10.0.19045	i7 10 th gen, 4 core	16 GB	SSD 512 GB
Sofia Maule	Windows 11	i5-1035G1	8 GB	SSD 256 GB

4. Results and Discussion

4.1. Runs Description

In this section, we provide a brief description of the runs produced (runs used just for parameter tuning are omitted), along with the corresponding run identifiers used later in the performance evaluation. All runs were executed using the `BM25Similarity` scoring function, they rely on the French and English documents collection, with filters and preprocessing steps as described in *Subsection 2.2*, and use stoplists `english_stopwords.txt` and `french_stopwords.txt`, unless otherwise specified.

All initial runs employed a `BooleanQuery` strategy during the search phase, aimed at evaluating the effectiveness of different analyzers:

- **stdAnaBool**: the run was performed using Lucene’s StandardAnalyzer for both the English and French collections;
- **minAnaBool1**: uses FrenchMinimalStemFilter and EnglishMinimalStemFilter for stemming;
- **liminAnaBool**: combines FrenchLightStemFilter stemming for French text and EnglishMinimalStemFilter stemming for English text;
- **snowAnaBool**: applies SnowballFilter stemming for both languages;
- **porterAna1Bool**: applies FrenchLightStemFilter stemming for French and PorterStemFilter stemming for English.

Once the best analyzer configuration was identified as the one used in minAnaBool1, we retained it for all subsequent runs. Additionally, all following runs used the index built with document-level language detection, varying the query types, the number of retrievable documents, and the applied query manipulations:

- **minAnaBool300**: uses Boolean queries, retrieves at most 300 documents per topic;
- **minAnaBool1000**: uses Boolean queries, retrieves at most 1000 documents per topic;
- **minAnaPhr300**: uses Phrase queries, retrieves at most 300 documents per topic;
- **minAnaBool1000_GPT**: uses Boolean queries, retrieves at most 1000 documents per topic, with GPT-processed queries;
- **minAnaPhr300_GPT**: uses Phrase queries, retrieves at most 300 documents per topic, with GPT-processed queries;
- **minAnaBool1000_GPT_QEDict**: uses Boolean queries, retrieves at most 1000 documents per topic, with GPT-processed queries, dictionary-based query expansion;
- **minAnaPhr300_GPT_QEDict**: uses Phrase queries, retrieves at most 300 documents per topic, with GPT-processed queries, dictionary-based query expansion;
- **minAnaMixed**: uses combination of Boolean (boost 0.9) and Phrase (boost 2.0) queries, retrieves at most 1000 documents per topic (best model without GPT-processed queries);
- **minAnaMixed3_GPT**: uses combination of Boolean (boost 0.9) and Phrase (boost 1.5) queries, retrieves at most 1000 documents per topic, with GPT-processed queries (best model).

4.2. Runs Analysis

We present here a table summarizing the performance of all the systems described in *Subsection 4.1*, reporting both MAP and nDCG scores. Given the issues that particularly affected the early months of the collection, we provide average values both for the entire collection and specifically for the period from October 2022 to February 2023. Systems highlighted in bold are those selected for submission to the LongEval challenge.

4.3. Statistical Analysis

This section presents the comparative evaluation of system performance across different retrieval configurations and query strategies. The analysis focuses on the metrics introduced in *Subsection 3.2*: **MAP!**, **nDCG!**, and interpolated precision at standard recall levels (**iPrec@Recall**), which provide complementary insights into retrieval effectiveness.

All visualizations were generated using the automated MATLAB framework described in *Subsection 3.2.1*. For each group of runs, we typically include two plots per metric: one showing mean performance across months or recall levels, and one with standard deviation to reflect variability. However, in some cases, we opted to display only the mean plots when the additional variance information did not offer significant interpretive value or would have unnecessarily cluttered the presentation. The omitted standard deviation plots are available in the appendix for completeness.

Table 2

Performance of the runs

Run	MAP	nDCG	MAP (Oct-Feb)	nDCG (Oct-Feb)
minAnaMixed3_GPT	0.1715	0.2631	0.2111	0.3089
minAnaMixed	0.1657	0.2541	0.2046	0.2993
minAnaPhr300	0.1487	0.1946	0.1825	0.2316
minAnaPhr300_GPT	0.1479	0.1962	0.1800	0.2310
minAnaBool1000_GPT	0.1471	0.2378	0.1826	0.2804
minAnaBool1000_GPT_QEDict	0.1441	0.2343	0.1792	0.2769
minAnaBool1000	0.1333	0.2206	0.1647	0.2594
minAnaBool300	0.1330	0.2086	0.1643	0.2469
minAnaBool1	0.1274	0.2027	0.1580	0.2402
porterAna1Bool	0.1267	0.2018	0.1563	0.2384
liminAnaBool	0.1260	0.2011	0.1554	0.2375
snowAnaBool	0.1259	0.2008	0.1554	0.2373
minAnaPhr300_GPT_QEDict	0.1147	0.1564	0.1388	0.1827
stdAnaBool	0.1010	0.1675	0.1244	0.1972

4.3.1. Boolean Runs

We begin our analysis with the group of runs based on the `BooleanQuery` setting, introduced in Subsection 2.5 and described in detail in Subsection 4.1. This subset serves as a consistent baseline for comparing different analyzer configurations under a fixed query model.

The evaluation relies on the standard metrics presented in Subsection 3.2, and all plots were produced using the MATLAB-based analysis tools discussed in Subsection 3.2.1. For each metric (**MAP!**, **nDCG!**, and **iPrec@Recall**) we include one plot showing average performance over time or recall levels, and one with grouped error bars to highlight variability. This layout enables both comparative and stability-focused observations.

The results for **MAP!** are shown in Figures 1 and 2, those for **nDCG!** in Figures 3 and 4, and those for **iPrec@Recall** in Figures 5 and 6.

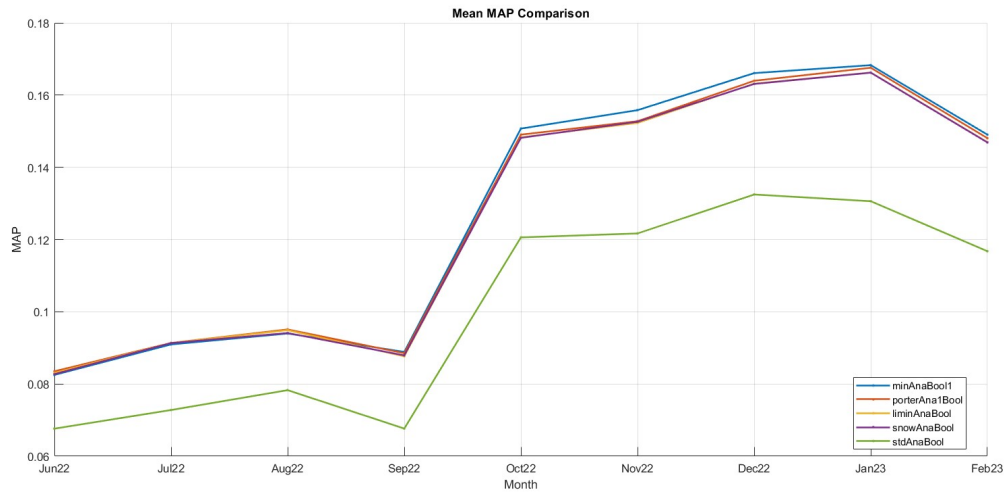


Figure 1: Mean **MAP!** over time for runs using `BooleanQuery`.

From the analysis of the Boolean runs, several trends emerge. As shown in Figures 1 and 3, all systems exhibit a noticeable improvement in both **MAP!** and **nDCG!** starting in October 2022. This pattern is consistent with our earlier observations (see Subsection 3.1), where we noticed some irregularities in the relevance judgments during the initial months. Although we cannot draw firm conclusions about their

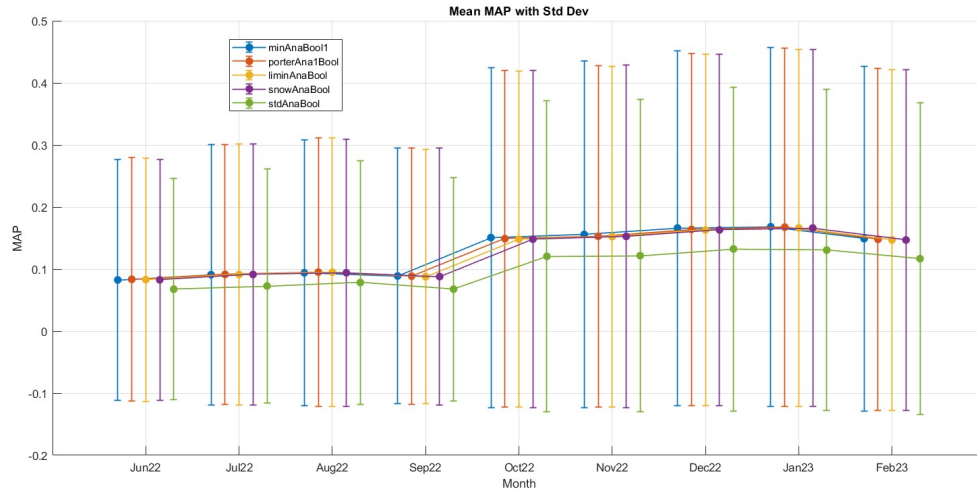


Figure 2: Mean MAP! with standard deviation for BooleanQuery runs. Error bars indicate the variability of performance across queries for each month.

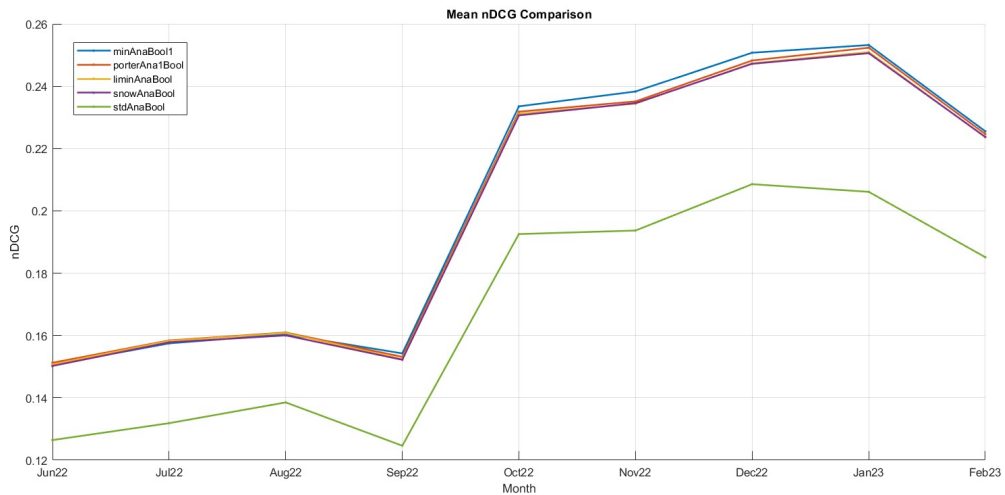


Figure 3: Mean nDCG! over time for runs using BooleanQuery.

cause, the improved separation of system performance from October onward suggests a stabilization in evaluation conditions.

Among the tested configurations, minAnaBool1 consistently achieved the highest scores in the later months, indicating the effectiveness of analyzers that apply light, language-aware stemming. stdAnaBool, based on Lucene’s default analyzer, delivered comparatively lower scores but serves as a strong and reliable baseline for assessing the impact of more customized or linguistically informed approaches.

The standard deviation plots (Figures 2 and 4) indicate that variability across queries remains consistently high throughout the evaluation period, with no clear temporal trend or major differences between systems in terms of dispersion. This means that, although the differences in average performance are attributable to the systems themselves, the high query-level variability should be kept in mind when interpreting the results.

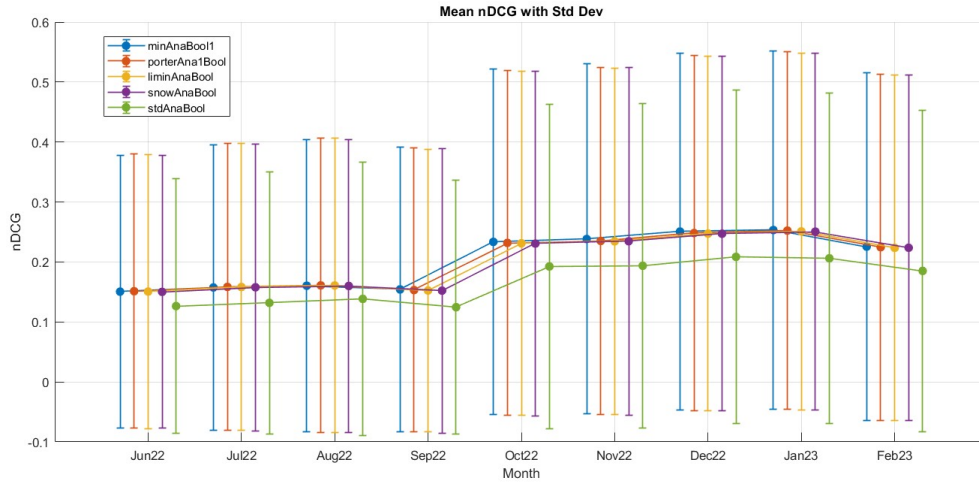


Figure 4: Mean nDCG@1 with standard deviation for BooleanQuery runs. Error bars indicate the variability of performance across queries for each month.

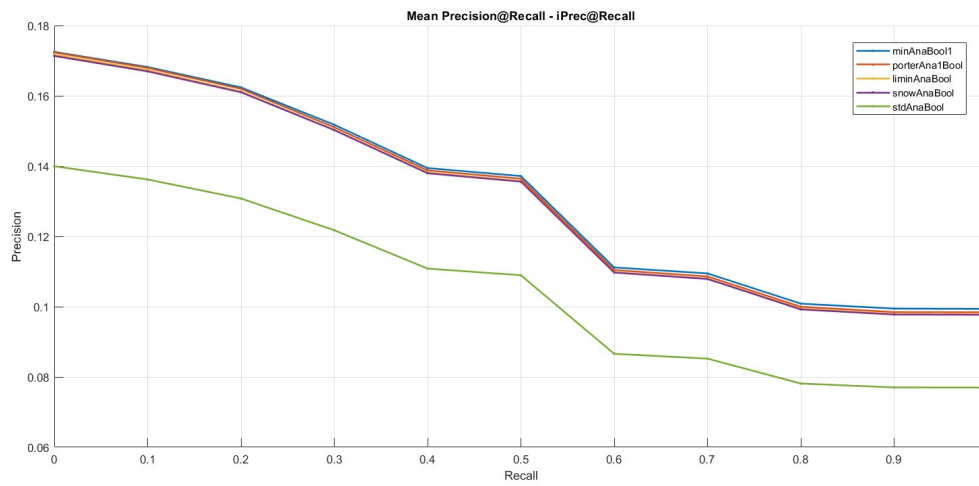


Figure 5: Mean iPrec@recall over time for runs using BooleanQuery.

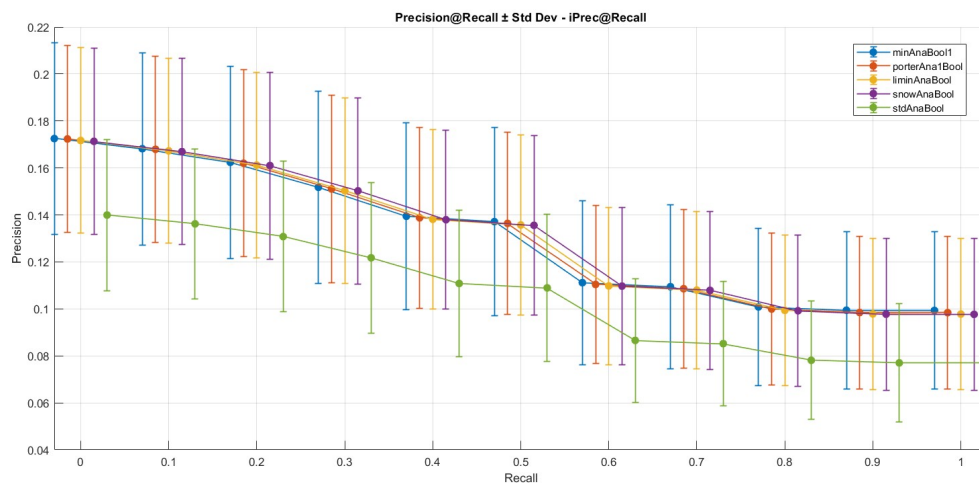


Figure 6: Mean iPrec@recall with standard deviation for BooleanQuery runs. Error bars indicate the variability of performance across queries for each month.

Ultimately, the plots (Figures 5 and 6) provide further confirmation of the trends discussed above. The minAnaBool1 system consistently produced higher precision across all recall levels compared to the other configurations. The error bars in the second plot show that variability remains consistently high throughout the recall spectrum, reinforcing the importance of interpreting average precision values with caution. Despite this, stdAnaBool stands out with lower performance across nearly all recall levels, confirming its usefulness as a stable baseline for comparison. Overall, the iPrec@Recall plots offer a complementary perspective on system behavior and help validate the relative ranking observed in the other evaluation metrics.

4.3.2. Phrase vs Document Language Detection

This subsection compares two runs described in Subsection 4.1, minAnaBool1 (sentence-level language detection) and minAnaBool1300 (document-level detection), to assess the impact of language detection granularity on retrieval performance.

We consider the same three evaluation metrics discussed in Subsection 3.2: **MAP!**, **nDCG!**, and **iPrec@Recall**, reporting only mean values as per the approach in Subsection 4.3.

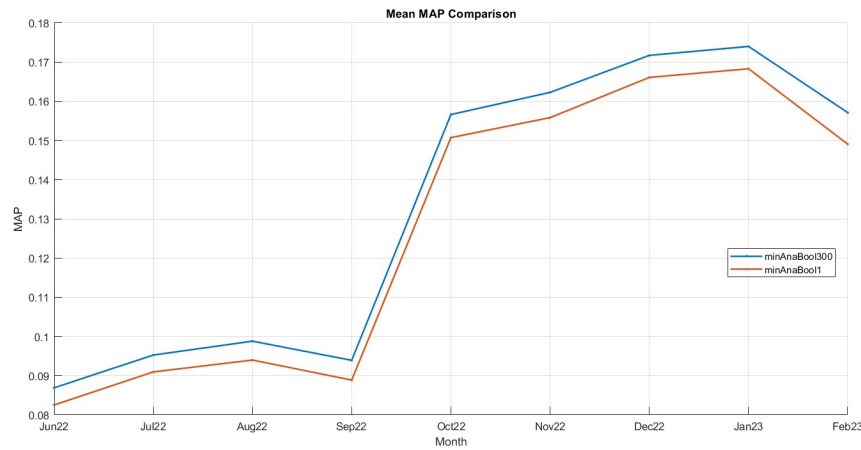


Figure 7: Mean **MAP!** over time comparing minAnaBool1 (sentence-level detection) and minAnaBool1300 (document-level detection).

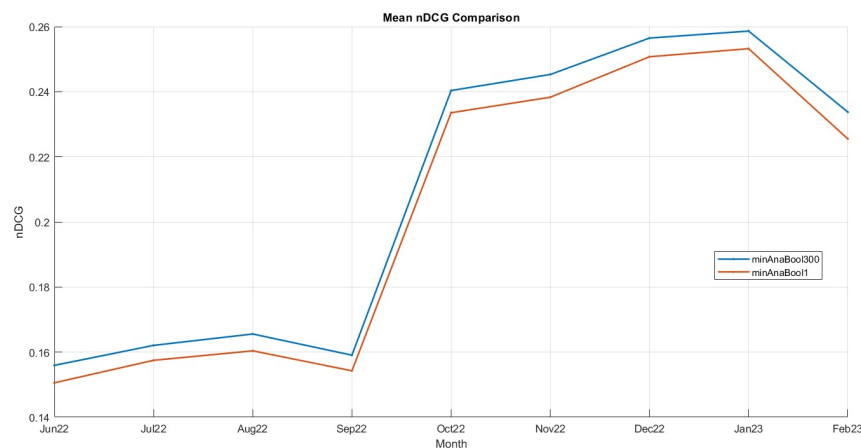


Figure 8: Mean **nDCG!** over time for minAnaBool1 and minAnaBool1300.

Overall, minAnaBool1300 consistently outperforms minAnaBool1 across all evaluation metrics, indicating that document-level language detection is more effective in this context. Although not shown

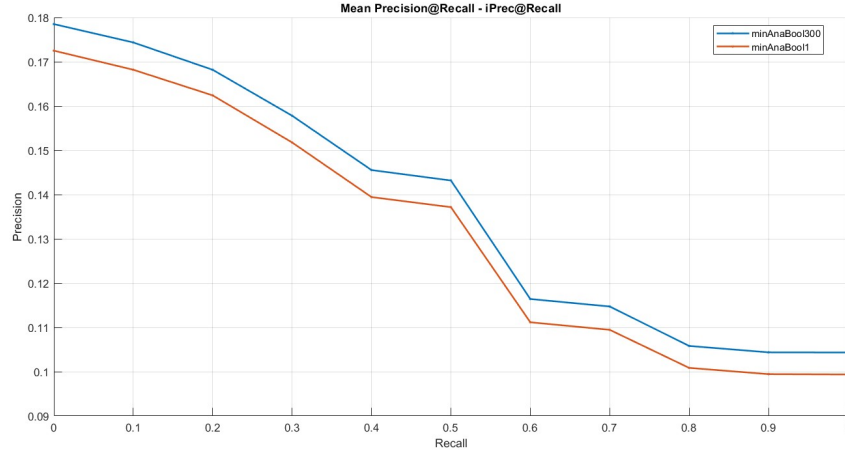


Figure 9: Mean interpolated precision at standard recall levels (iPrec@Recall) for minAnaBool1 and minAnaBool300.

here, standard deviation remains consistently high; corresponding variability plots are available in Appendix ??.

4.3.3. Boolean vs Phrase Queries

In this subsection, we evaluate the effect of switching from Boolean queries to Phrase queries during the search phase. The comparison involves two systems presented in *Subsection 4.1*: minAnaBool300, which performs Boolean matching, and minAnaPhr300, which uses phrase-based retrieval.

We focus on the three evaluation metrics discussed in *Subsection 3.2*: **MAP!**, **nDCG!**, and **iPrec@Recall**. We report only mean values here; standard deviation plots are available in Appendix ??.

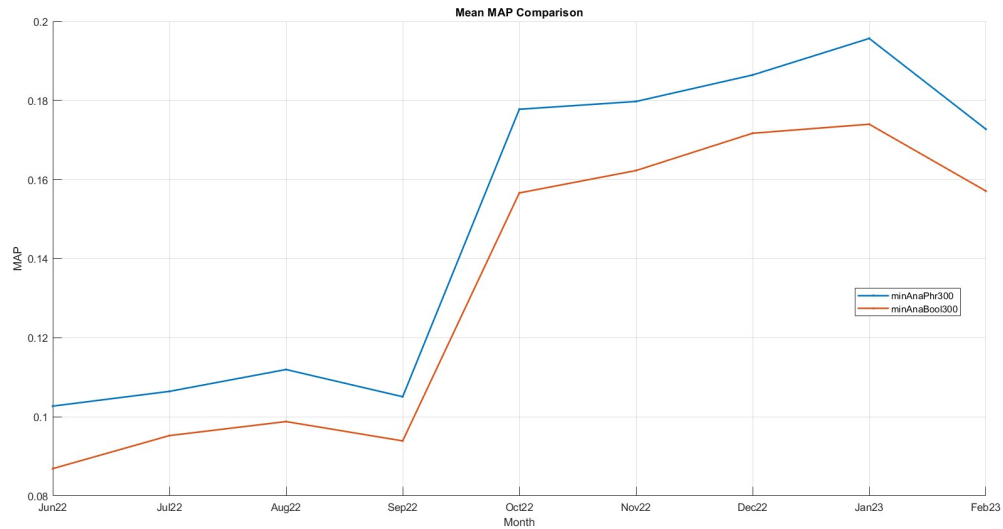


Figure 10: Mean **MAP!** over time comparing minAnaBool300 (Boolean query) and minAnaPhr300 (Phrase query).

Overall, the plots shows that minAnaPhr300, which uses PhraseQuery, has higher scores for **MAP!** but lower values for **nDCG!**. The iPrec@Recall plot, suggests that minAnaPhr300 is better for precision-oriented tasks, while minAnaBool300 is more indicated for recall-oriented systems. While the standard deviation remains consistently high, the performance gaps are clearly visible across all

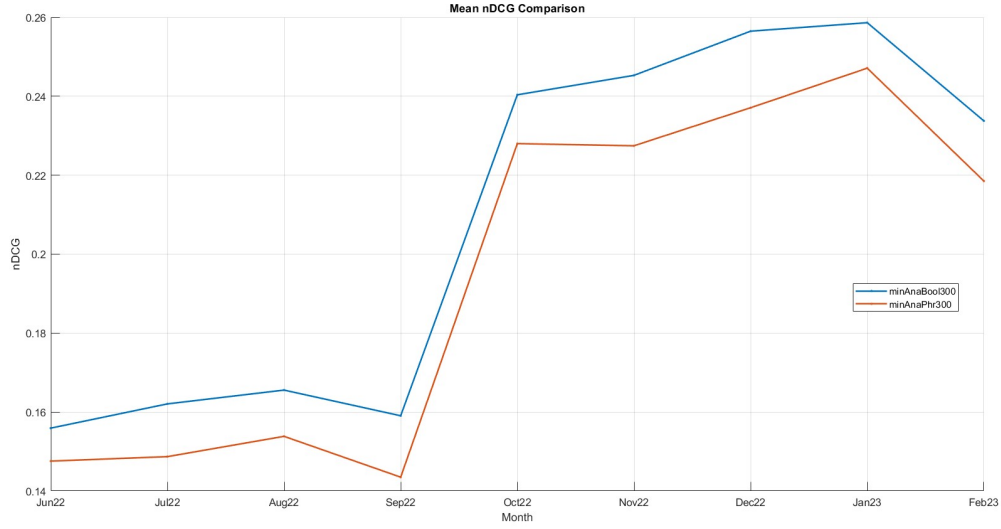


Figure 11: Mean $nDCG!$ over time comparing minAnaBoo1300 and minAnaPhr300.

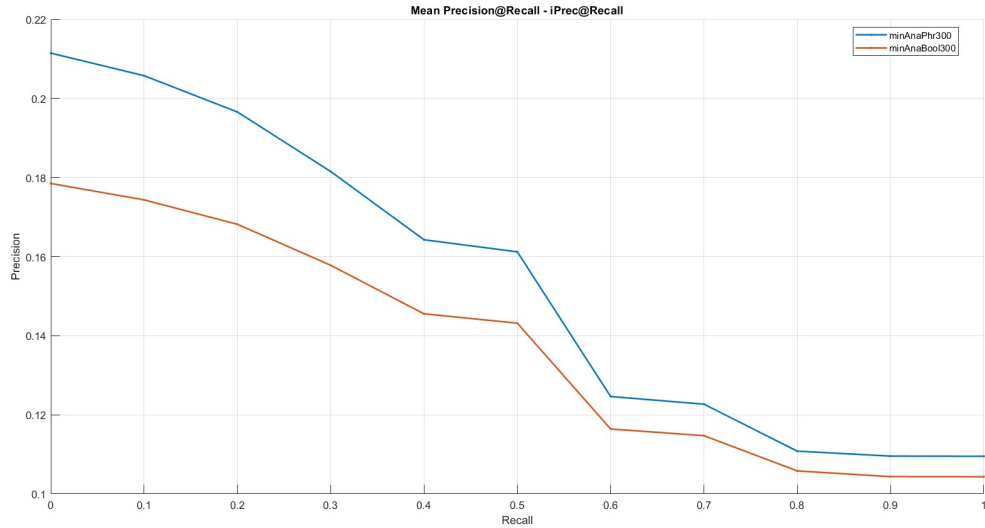


Figure 12: Mean interpolated precision at standard recall levels (iPrec@Recall) for minAnaBoo1300 and minAnaPhr300.

measures.

4.3.4. Query Manipulation and Query Type

In this analysis, we compare four configurations that combine two query strategies (Boolean vs Phrase) with and without query manipulation: minAnaBoo11000, minAnaBoo11000_GPT, minAnaPhr300, and minAnaPhr300_GPT, as described in *Subsection 4.1*. The goal is to evaluate how different query types respond to improved query formulations (achieved by pre-processing them with an LLM, as described in *Subsection 2.4.4*).

Figures 13, 14, and 15 present the results for $MAP!$, $nDCG!$, and $iPrec@Recall$. As outlined in *Subsection 4.3*, only the mean plots are included here; standard deviation plots are provided in Appendix ???. This comparison shows that query manipulation has varying effects depending on the query type used

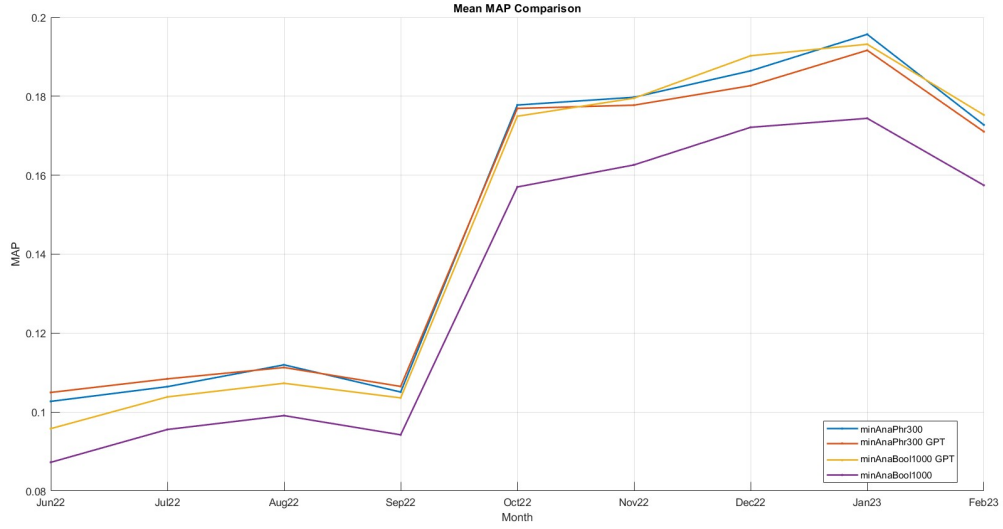


Figure 13: Mean **MAP!** over time for the four configurations combining query type and manipulation.

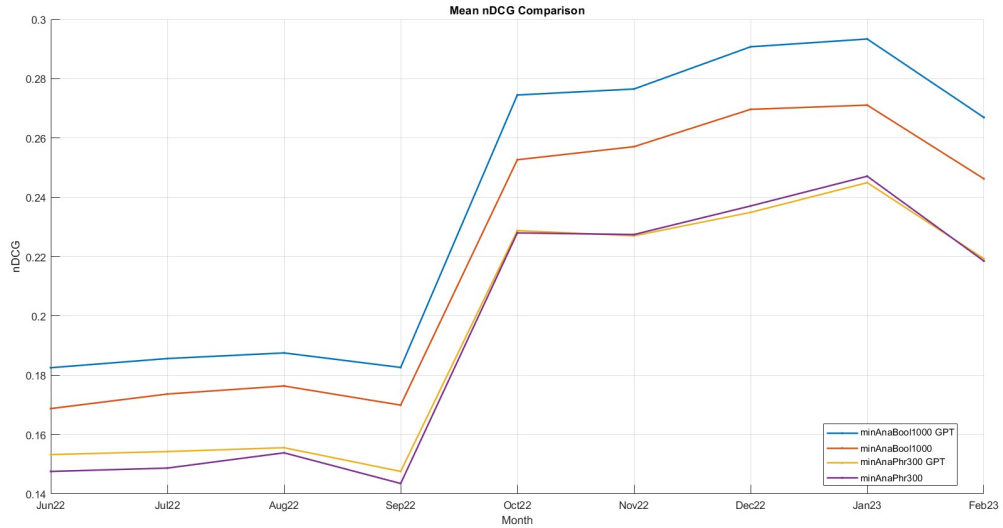


Figure 14: Mean **nDCG!** over time for the four configurations combining query type and manipulation.

during searching. In particular, minAnaPhr300 achieves slightly better performance than minAnaPhr300_GPT, suggesting that query manipulation is counterproductive for phrase-based queries. For Boolean queries, the opposite is true: minAnaBoo1000_GPT significantly outperforms minAnaBoo1000 across all three metrics.

4.3.5. Query Expansion and Query Type

In this analysis, we compare four configurations that combine two query strategies (Boolean vs Phrase) with and without query expansion: minAnaBoo1000_GPT, minAnaBoo1000_GPT_QEDict, minAnaPhr300_GPT, and minAnaPhr300_GPT_QEDict, as described in *Subsection 4.1*. The goal is to evaluate how different query types respond to the expansion of queries using a dictionary-based synonyms generator, as described in *Subsection 2.4.5*.

Figures 16, 17, and 18 present the results for **MAP!**, **nDCG!**, and **iPrec@Recall**. As outlined in *Sub-*

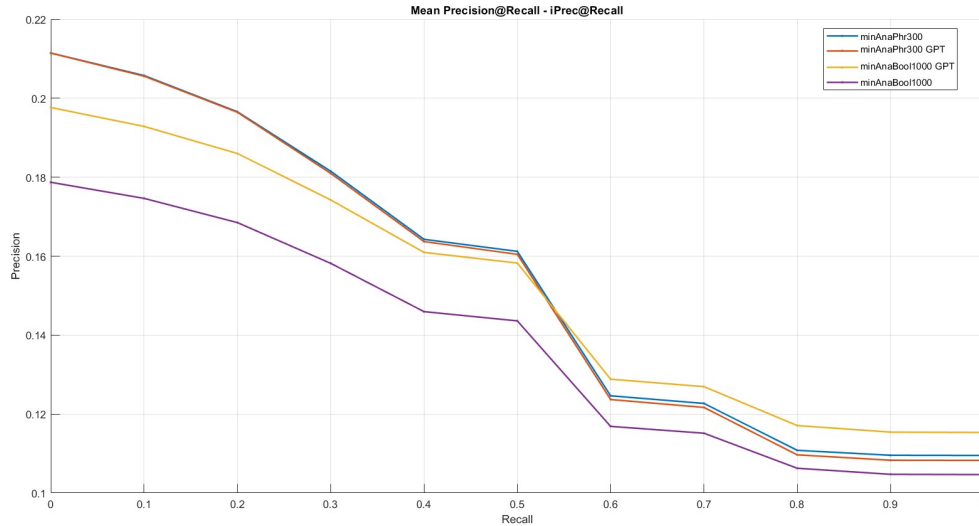


Figure 15: Mean interpolated precision at standard recall levels (iPrec@Recall) for the four configurations.

section 4.3, only the mean plots are included here; standard deviation plots are provided in Appendix ??.

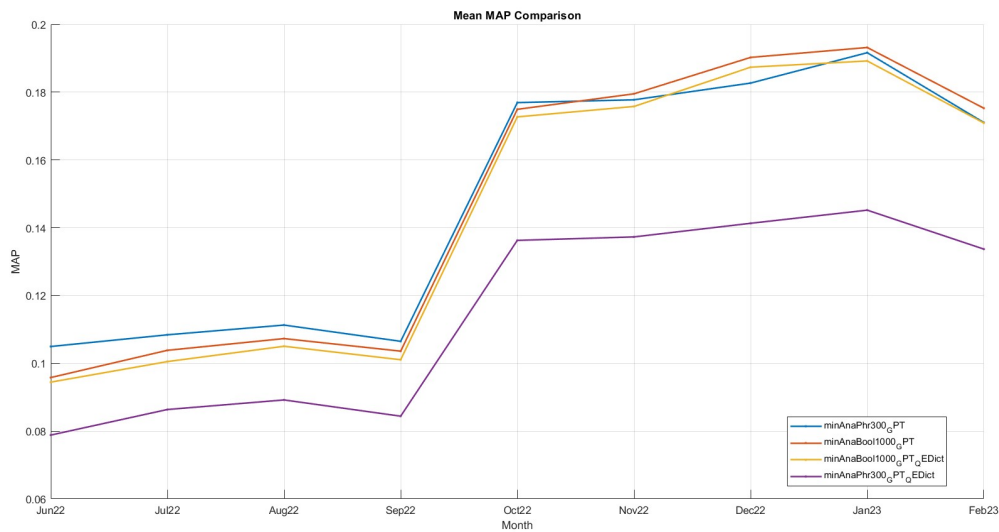


Figure 16: Mean MAP! over time for the four configurations combining query type and query expansion.

This comparison shows that our dictionary-based query expansion reduces the performance of both query types, with a clear decline across all three metrics for phrase-based queries.

4.3.6. Hybrid Query Type

In this analysis, we compare the two previously used query strategies (Boolean and Phrase, using the best system for each) with a hybrid approach that combines them: minAnaBool1000_GPT, minAnaPhr300, minAnaMixed, and minAnaMixed3_GPT, as described in Subsection 4.1. The goal is to evaluate whether combining two distinct query types can enhance performance by leveraging the strengths of each original model.

Figures 19, 20, and 21 present the results for MAP!, nDCG!, and iPrec@Recall. As outlined in Subsection 4.3, only the mean plots are included here; standard deviation plots are provided in Appendix ??.

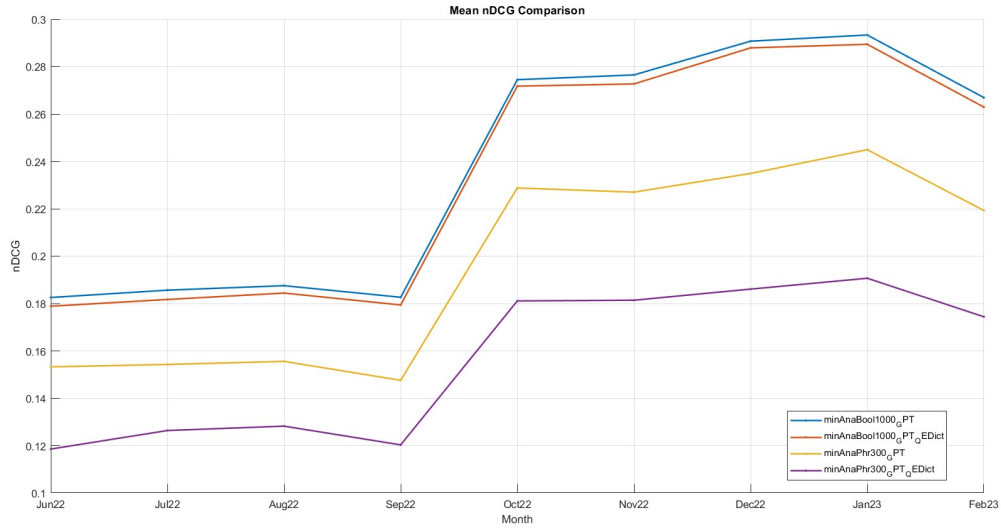


Figure 17: Mean **nDCG!** over time for the four configurations combining query type and query expansion.

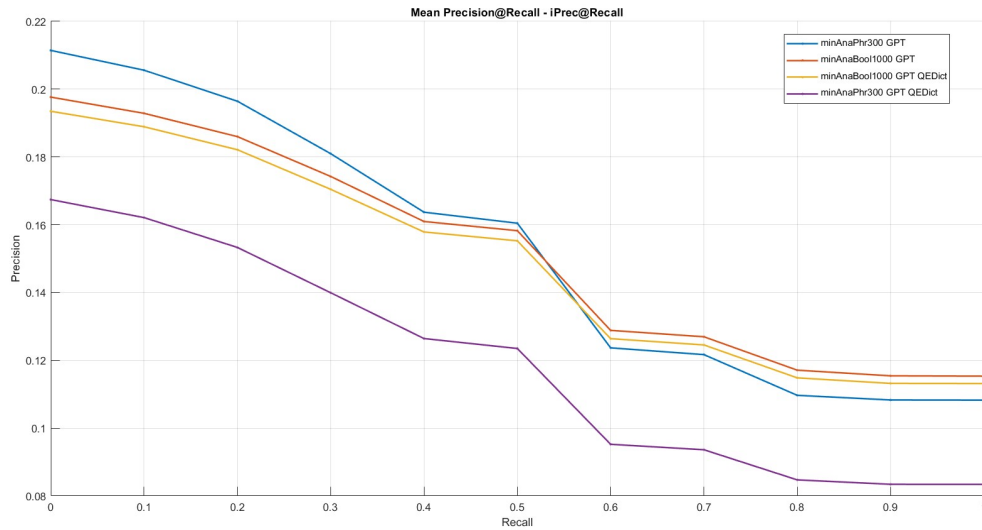


Figure 18: Mean interpolated precision at standard recall levels (**iPrec@Recall**) for the four configurations.

This comparison shows that combining Boolean and Phrase queries yields better scores across all three metrics. By adjusting the boost values for the combination, we achieved our best-performing systems both with GPT-processed queries (**minAnaMixed3_GPT**) and without them (**minAnaMixed_GPT**).

4.3.7. Boolean Runs – Updated Queries

On May 6th, the LongEval organizers released an updated set of queries and qrels to address some of the issues previously observed in the initial data. The graphs presented in this subsection report the same evaluation metrics used in our earlier analysis, now applied to runs performed with the updated inputs.

The systems and configurations are identical to those described in Section 4.3.1; the only difference is that they were executed using the revised queries and qrels provided.

Figures 22, 23, and 24 present the results for **MAP!**, **nDCG!**, and **iPrec@Recall**. As outlined in Sub-

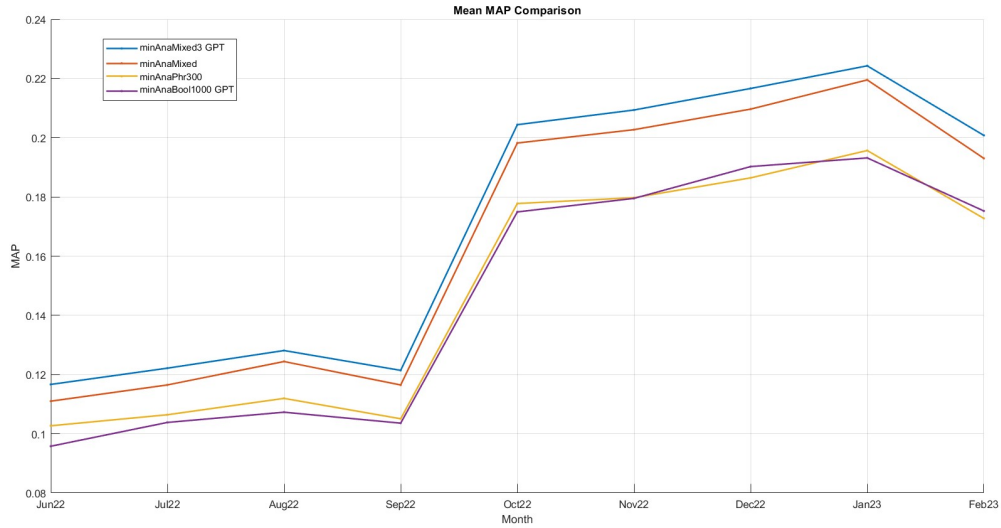


Figure 19: Mean **MAP!** over time for the four systems compared.

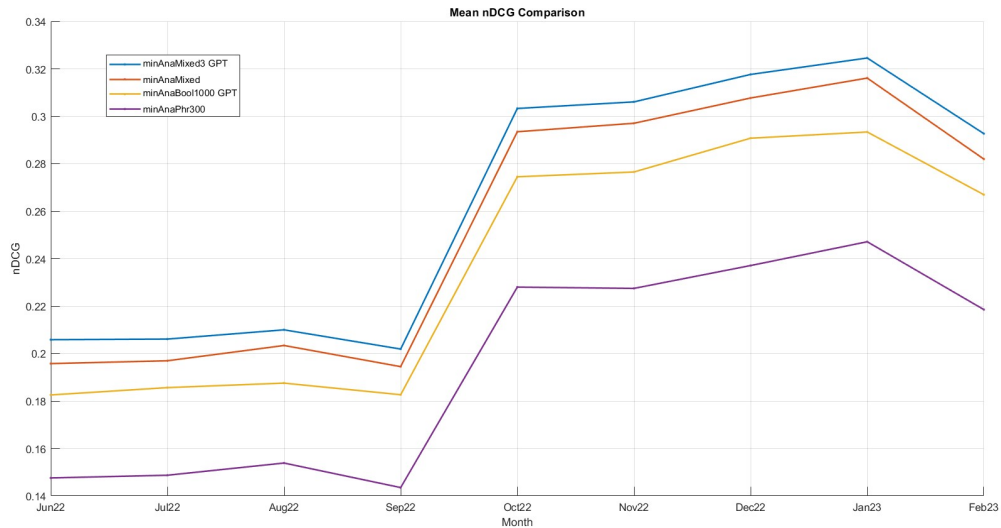


Figure 20: Mean **nDCG!** over time for the four systems compared.

section 4.3, only the mean plots are included here; standard deviation plots are provided in Appendix ??.

To complement this analysis, we also include ANOVA-based multiple comparison plots for **MAP!** (Figure 25) and **nDCG!** (Figure 26). These plots help highlight which performance differences between systems are statistically significant.

Overall, the relative behavior and ranking of the systems remain consistent with those observed in Section 4.3.1. The only notable difference is a general increase in scores across all metrics, suggesting that the updated queries and qrels led to an overall improvement in measured performance.

From the ANOVA plots (Figures 25 and 26), it is evident that `stdAnaBool_NewQuery` performs significantly worse than all other systems in both **MAP!** and **nDCG!**. The remaining systems show very similar performance and are not statistically distinguishable from one another, reinforcing the consistency of results observed in the mean trends.

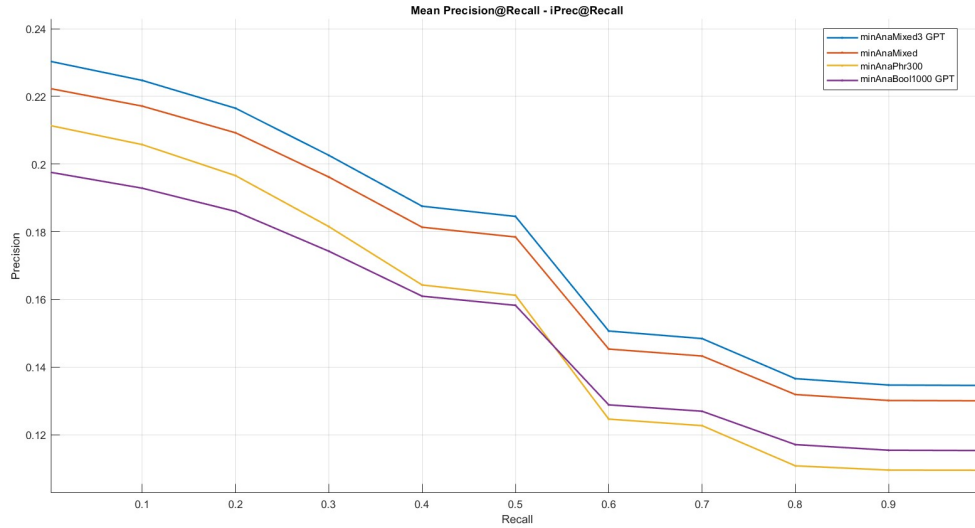


Figure 21: Mean interpolated precision at standard recall levels (iPrec@Recall) for the four systems compared.

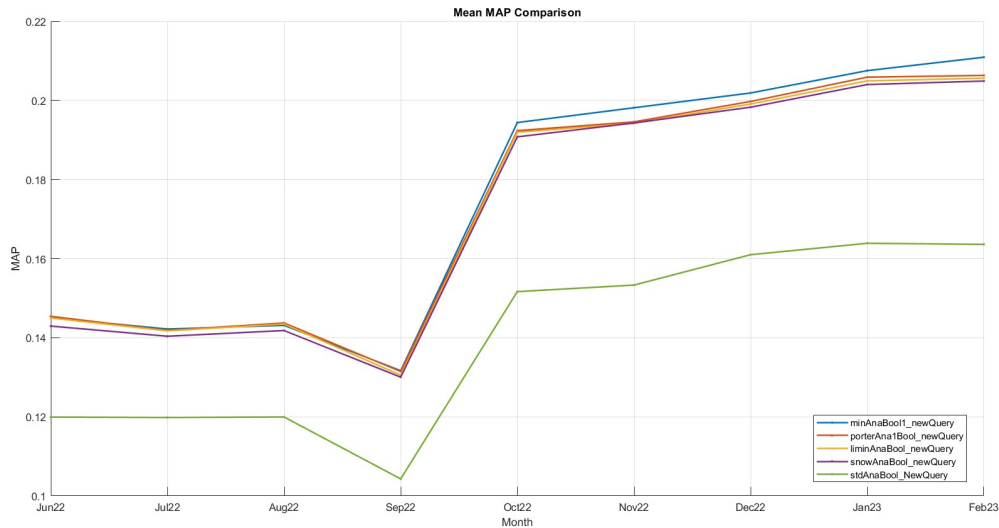


Figure 22: Mean MAP! over time for the systems using different analyzers.

4.3.8. Min Analyzer Variants – Updated Queries

This subsection presents the performance of all our system variants built using the MinAnalyzer, evaluated with the updated set of queries and qrels released on May 6th. The goal is to compare different combinations of indexing and querying strategies within the same analyzer family, which previously proved to be the most effective.

The evaluation relies on the same metrics as in previous sections: **MAP!**, **nDCG!**, and **iPrec@Recall**. Figures 27, 28, and 29 report the results. Standard deviation plots are available in Appendix ??.

We also include ANOVA-based multiple comparison plots (Figures 30 and 31) to assess the statistical significance of performance differences among systems.

Overall, the relative ranking and behavior of the systems are consistent with previous observations. The main difference lies in a uniform increase in scores across all metrics, reflecting the impact of the revised query and relevance data.

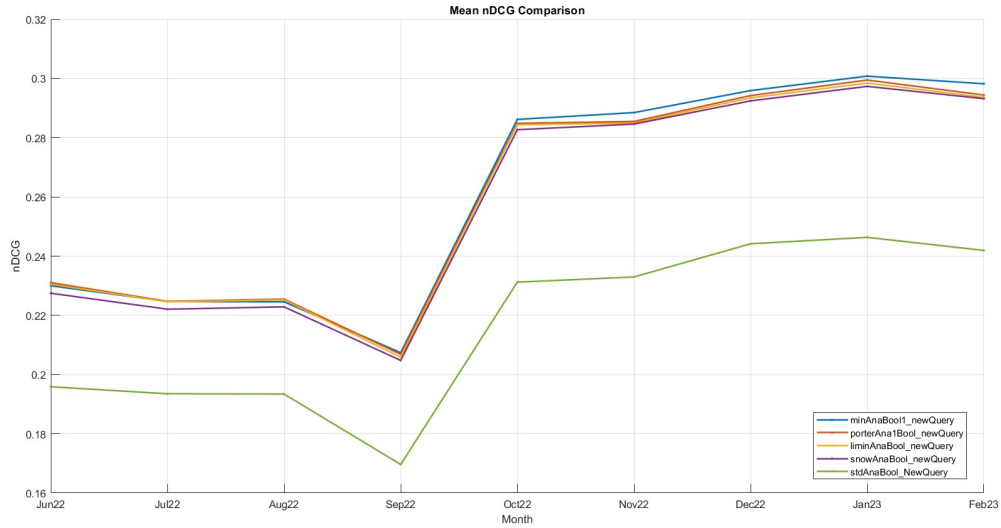


Figure 23: Mean **nDCG!** over time for the systems using different analyzers.

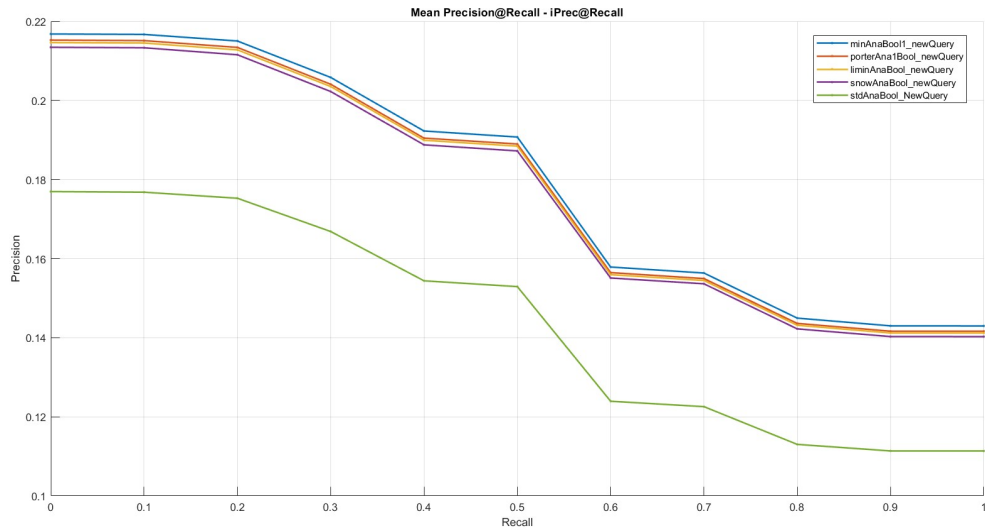


Figure 24: Mean interpolated precision at standard recall levels (**iPrec@Recall**) for the systems using different analyzers.

The ANOVA plots confirm these trends, showing that `minAnaMixed3_GPT_newQuery` and `minAnaMixed_newQuery` are statistically the best-performing systems across both metrics. These are the only configurations that form a clearly distinct group in the multiple comparison results.

It is also worth highlighting the behavior of `minAnaBool1000_GPT_newQuery` and `minAnaBool1000_GPT_QEDict_` which show divergent results depending on the metric considered. In terms of **MAP!**, both systems fall well below the upper group, indicating a noticeable performance gap. However, when evaluated with **nDCG!**, they appear in the same statistical group as `minAnaPhr300_newQuery` and `minAnaPhr300_GPT_newQuery`, suggesting greater robustness in ranking quality than in overall precision.

Finally, `minAnaBool1000_newQuery` presents an interesting case: it is part of the lowest-performing group in terms of **MAP!**, yet in **nDCG!**, it forms a separate group that is statistically distinct from the bottom tier, indicating a comparatively better performance when considering ranking effectiveness rather than strict retrieval completeness.

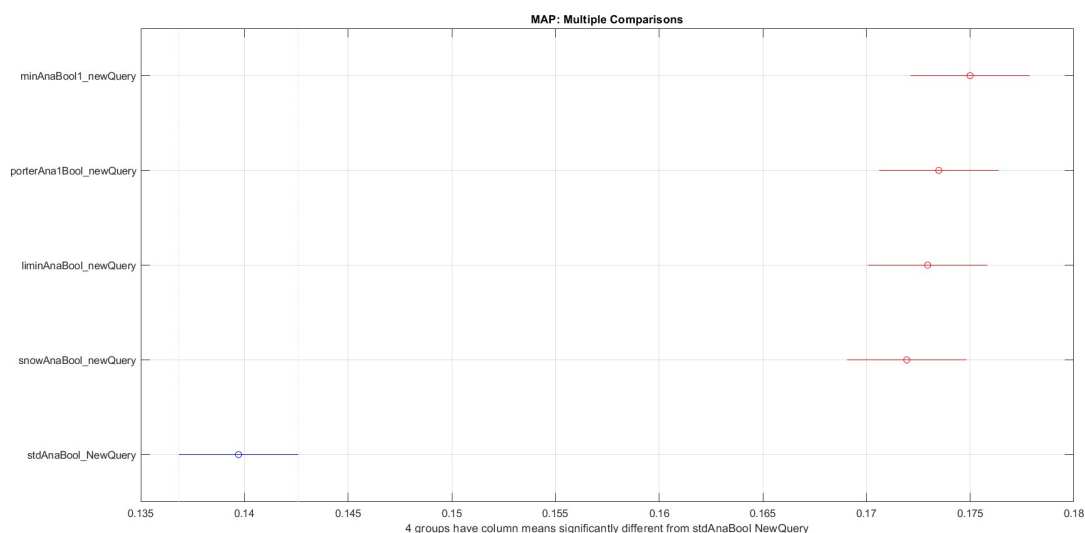


Figure 25: Multiple comparison results for **MAP!** based on two-way ANOVA.

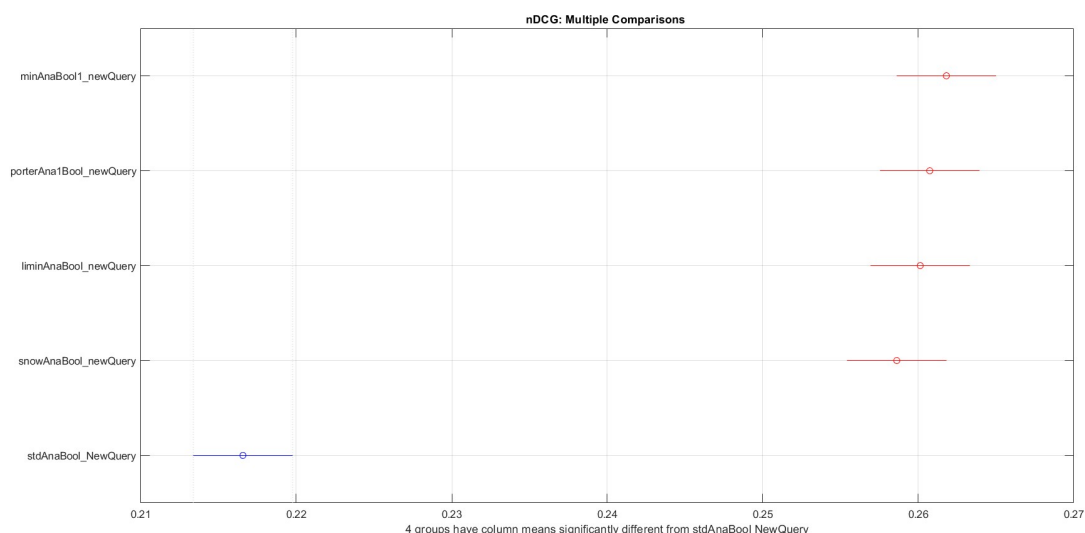


Figure 26: Multiple comparison results for **nDCG!** based on two-way ANOVA.

5. Conclusions and Future Work

5.1. Conclusion

Throughout the project, we observed that even light query processing using GPT to correct errors in queries led to significant improvements when used on Boolean models. We found that combining Boolean queries with Phrase queries outperformed each method individually, confirming the added value of this hybrid approach. As we progressed through the various steps of development, we consistently saw improvements in retrieval performance.

However, query expansion, especially when using dictionaries, did not yield the expected benefits. We concluded that this approach was too generic, particularly for French, where available resources, such as WordNet for English, are not as rich. The limitations of French language tools ultimately led to suboptimal results in the query expansion process.

Overall, the project demonstrated that even small enhancements in query handling can lead to substantial

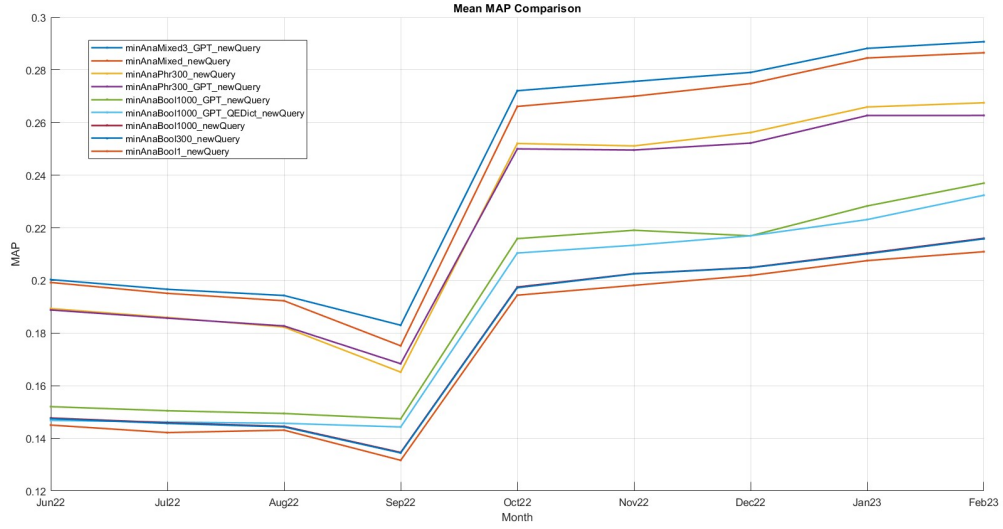


Figure 27: Mean MAP! over time for all systems using the Min analyzer.

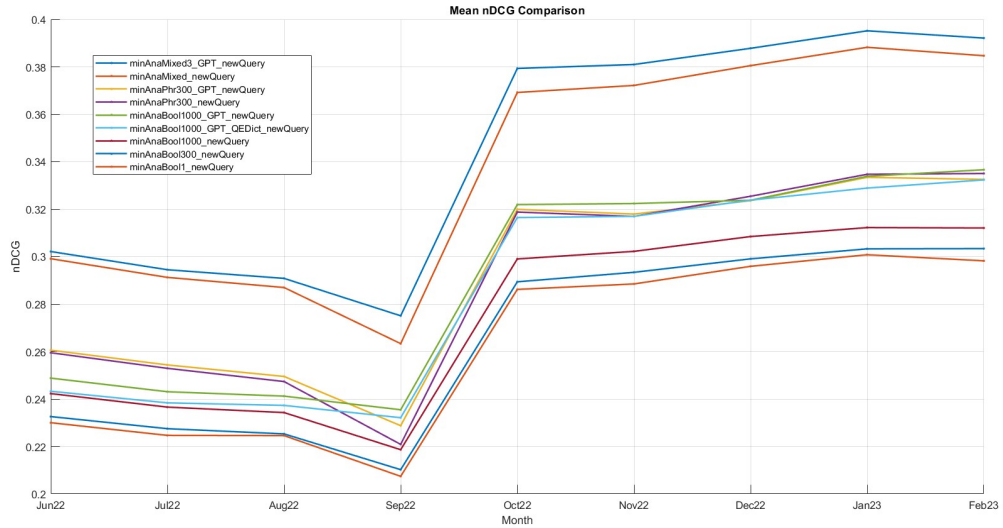


Figure 28: Mean nDCG! over time for all systems using the Min analyzer.

gains in retrieval performance, but the choice of tools and approaches, especially for less-resourced languages like French, remains crucial for achieving optimal results.

5.2. Future work

Future work will focus on incorporating more advanced retrieval techniques that could not be explored in the current project due to hardware constraints and limitations in the available data. In particular, the use of *LLM!* (*LLM!*)s for tasks such as query rewriting, expansion, and contextual understanding represents a promising direction. These models can enhance retrieval effectiveness by generating more semantically rich and intent-aware queries or by providing improved ranking signals during post-processing.

To improve ranking quality, we attempted to integrate a reranking component into our retrieval pipeline. As mentioned before, given the inconsistencies observed in the provided qrels (particularly in the earlier

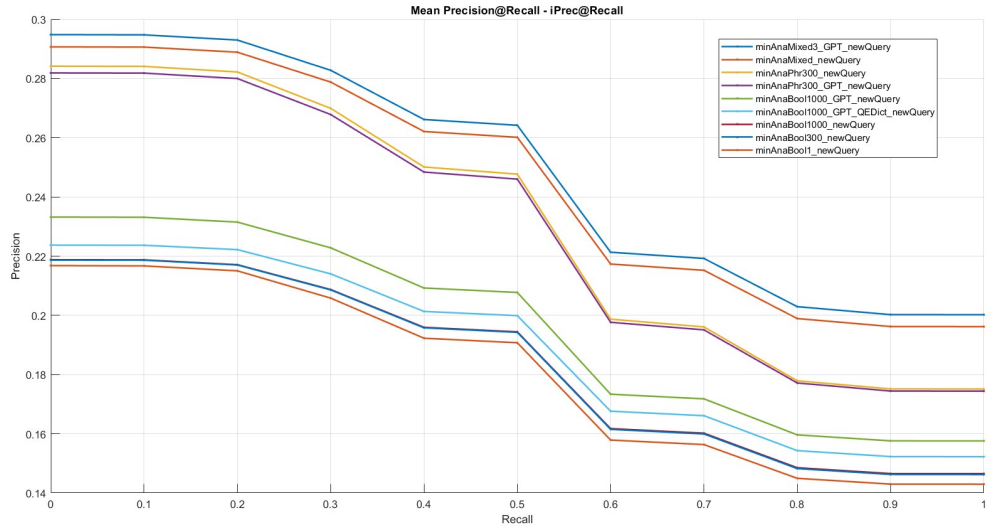


Figure 29: Mean interpolated precision at standard recall levels (iPrec@Recall) for all systems using the Min analyzer.

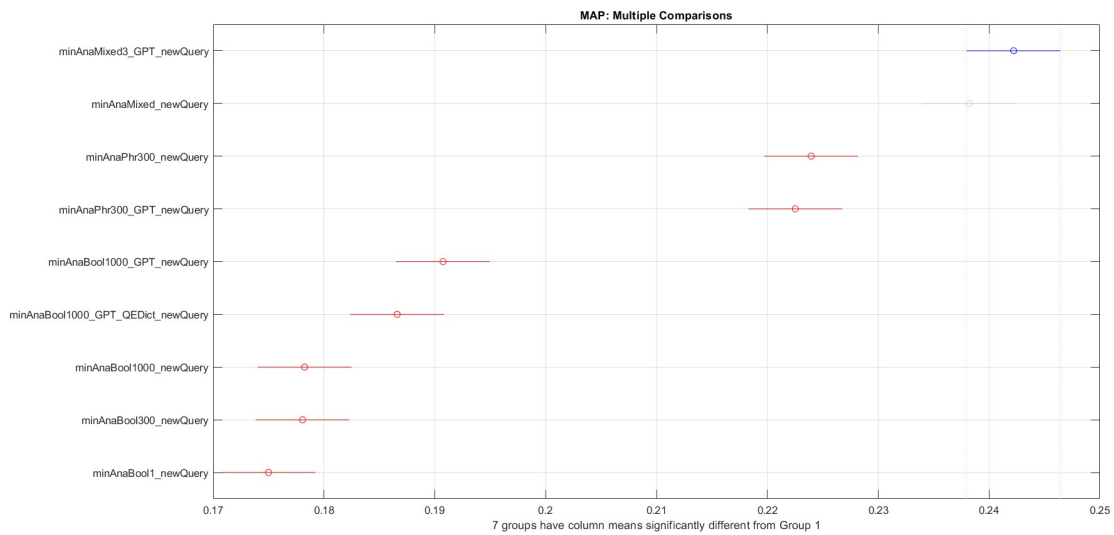


Figure 30: Multiple comparison results for **MAP!** across Min analyzer variants based on two-way ANOVA.

months of the dataset) we decided to adopt a soft reranking strategy based on semantic similarity using a pretrained deep learning model, rather than a supervised Learning to Rank approach, which could limit the feasibility of these approaches.

Due to hardware limitations, we were only able to test the module on a small subset of topics, which prevented a comprehensive evaluation of its effectiveness.

In future work, we plan to develop a more robust reranking module that not only includes a soft integration with BM25 scores, but is also capable of leveraging more advanced models. In particular, we intend to experiment with **LLM!**-based rerankers, which can model richer interactions between queries and documents and offer the potential for significantly better ranking performance, especially in cases of short or ambiguous queries.

Another avenue we intend to explore is the application of supervised Learning to Rank methods such as LambdaMART, which have shown strong performance in retrieval competitions and industry applications. However, the effectiveness of such models strongly depends on the availability of reliable

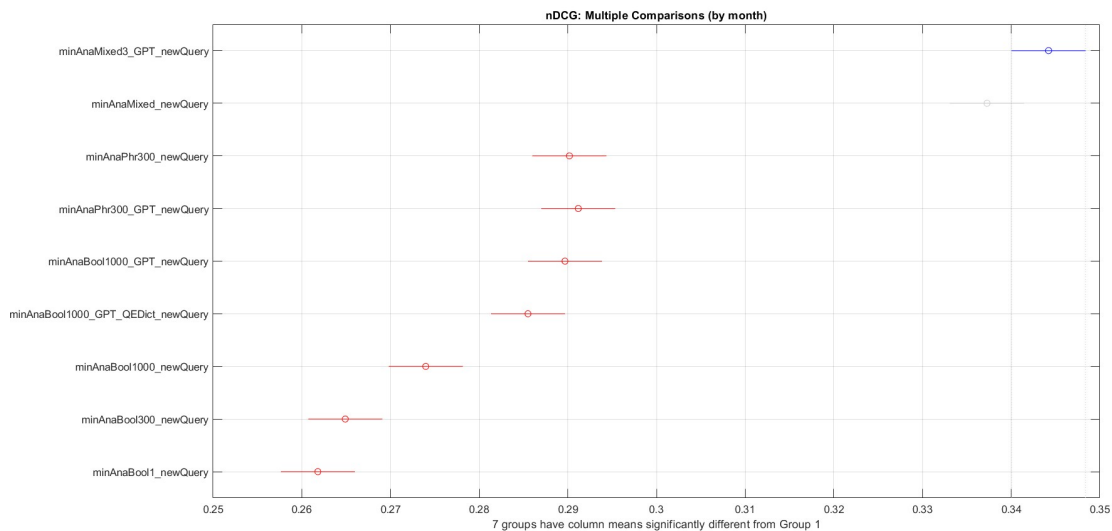


Figure 31: Multiple comparison results for **nDCG!** across Min analyzer variants based on two-way ANOVA.

and fine-grained relevance judgments.

Overall, advancing toward these techniques will require both a more robust computational infrastructure and improvements in data quality. Nonetheless, their integration could significantly push the boundaries of performance within the LongEval framework and similar longitudinal retrieval settings.

6. Group Members Contribution

Damiano Caon My primary focus in this project was the development of the `Searcher` class. My contributions mainly involved the implementation and enhancement of this component. I developed functionalities such as the configurability of the component, the management of `PhraseQuery` and `BooleanQuery` creation, and the parallelization of the search process to significantly reduce processing time. In addition, I worked on testing language detection mechanisms, conducting experiments with both a custom language detector `FrenchEnglishDetector` and the `Lingua`[4] library. I also explored the basic query expansion by integrating synonyms into queries. Furthermore, I made improvements to the `QueryParser` class to enhance the reading of the topic. Furthermore, I ran indexing and searching tests with the Porter stemmer configuration to evaluate its performance. For this reason, I've lastly implemented the `EvalGenerator` class, to rapidly process all the runs and build the needed `trec_eval` scores. Regarding this report, I have contributed to the compiling and the editing of the `Searcher` part in the Section 2.

Riccardo Dal Maschio I contributed to the development of several core components of the system. I implemented the `Parser` responsible for reading and cleaning the raw data, the `JsonDocument` class for document modeling, and the `QueryReader` utility for parsing queries. I also developed the `Main` class to manage the execution flow of indexing and searching, as well as utility classes such as `TimeFormatter` and `QrelsCleaner`, the latter used to remove duplicate entries from the `qrels` files. Additionally, I was responsible for setting up and integrating the `trec_eval` evaluation tool, compiling and configuring it to be immediately usable by the team without requiring manual installation (I did not produce the `evalGenerator` utility class).

On the documentation side, I set up the initial LaTeX template and wrote the first part of the Methodology section, including the `Parser` description, a first version of the Collection overview, and the Measures subsection within the Experimental Setup. I also wrote half of the Statistical Analysis section and was responsible for producing all the evaluation plots using MATLAB.

In addition to writing most of the MATLAB scripts used for evaluation, I also constructed the appendix and wrote the abstract included in this final report.

Alessandro Disarò I developed the `Indexer` class with a focus on performance improvements. I also implemented other key modules of the project, such as the custom language detector and the dictionary-based synonyms generator. Additionally, I created a Python script to generate synonyms using a local Flask server running an LLM. I conducted an in-depth analysis of the collection to understand the reasons behind our system's poor results, ultimately identifying the issue of misaligned qrels relative to the queries. I preprocessed the queries using GPT-4.0 Turbo, automating the workflow with a Python script. I carried out sequential testing across different systems to determine the best paths forward. For the report, I contributed the sections related to my developed components, the description of the runs, and the explanation of the experimental procedure.

Sofia Maule I focused on the development of the analyzers used in our IR pipeline. I implemented the `TestAnalyzer` class, which enabled us to flexibly test different preprocessing configurations through external JSON files. I created and maintained the associated stopword lists and parameter files to support experimentation across multiple stemming and filtering strategies. I also thought about reranking strategies and, based on the system constraints and available data, I implemented a soft reranking module. This included both the Java-side `HttpHelper` class and the Python-based `reranker_server.py` but due to hardware limitations, reranking could only be tested on a limited number of topics. I contributed to the initial evaluation of the analyzers and the execution of the first full system tests. Regarding the report, I authored the sections related to the analyzers and their corresponding run configurations.

References

- [1] CLEF LongEval, CLEF LongEval Tasks Overview, <https://clef-longeval.github.io/tasks/>, 2025. Accessed: 2025-04-19.
- [2] Oracle Java Magazine, Java JSON Serialization with Jackson, <https://blogs.oracle.com/javamagazine/post/java-json-serialization-jackson>, 2022. Accessed: 2025-04-19.
- [3] L. Bellin, A. A. Carè, M. Martini, M. T. Pepaj, M. Salvalaio, A. Segala, M. Tognon, N. Ferro, SEUPD@CLEF: Team GWCA on Longitudinal Evaluation of IR Systems by Using Query Expansion and Learning To Rank , in: Conference and Labs of the Evaluation Forum <https://ceur-ws.org/Vol-3497/paper-186.pdf>, 2023.
- [4] PeMISTAHL, Lingua: A Language Detection Library for Java and Kotlin, <https://github.com/pemistahl/lingua>, 2022. Accessed: 2025-04-19.
- [5] Princeton University, About WordNet, <https://wordnet.princeton.edu/>, 2010. Accessed: 2025-04-26.
- [6] B. Sagot, D. Fišer, Building a free french wordnet from multilingual resources, in: Ontolex 2008, Marrakech, Morocco, 2008.